# Pre-match football match outcome prediction from team form sequences

**Team of collaborators: Thodoris Grigoriou, Nikos Marios Spyropoulos, Michalis Mykoniatis**

**Course: Deep Learning**

# Research question

Does modeling a team's recent match sequence improve match outcome prediction compared with static rolling averages, and how much extra value comes from opponent context and event-derived features?

# Abstract

Football match outcome prediction is a challenging machine learning problem due to the dynamic nature of team performance, class imbalance, and the strong temporal dependencies present in sports data. This dissertation investigates the prediction of English Premier League match outcomes using a combination of feature engineering, Elo rating systems, classical machine learning methods, and deep learning architectures.

A complete end-to-end forecasting pipeline was developed, including data acquisition, preprocessing, leakage-free temporal splitting, team-centric feature construction, and sequence generation from historical match data. Multiple modeling approaches were evaluated, ranging from traditional machine learning algorithms to convolutional, recurrent, and hybrid neural network architectures designed to capture both short-term and long-term patterns in team performance.

The study further examines the impact of historical sequence length, Elo-based team strength features, and class imbalance handling strategies on predictive performance. Experimental findings indicate that carefully engineered features and team-strength representations play a crucial role in football forecasting, while increased model complexity does not necessarily translate into better predictive performance. The results highlight the importance of robust evaluation protocols and demonstrate the challenges associated with predicting balanced match outcomes, particularly draws.

Overall, this work provides a comprehensive comparison of modern machine learning and deep learning approaches for football outcome prediction and offers practical insights into the design of reliable sports forecasting systems.

# 1.Introduction

## 1.1 Motivation & Context

Football is widely considered one of the most unpredictable sports in the world. Unlike high-scoring games, a single goal—often the result of a random deflection, a refereeing decision, or momentary brilliance—can fundamentally alter the outcome of a 90-minute match. This high degree of variance makes football an excellent, albeit hostile, environment for applied machine learning.

Historically, sports forecasting relied heavily on team identities and raw historical win rates. However, modern predictive modeling demands a more granular understanding of form, momentum, and tactical matchups. The motivation for this project is to build a purely data-driven prediction engine. Rather than relying on the historical "reputation" of a club, the system forces models to evaluate matches strictly on recent performance metrics, underlying shot data, and relative momentum. By comparing traditional, flat-vector machine learning techniques against sequential deep learning models designed to understand the chronological flow of form, this project aims to uncover the mathematical limits of predicting human athletic competition using historical match data.

## 1.2 Problem Statement

The core task is framed as a multi-class classification problem. Given a sequence of recent historical match statistics for a Home Team and an Away Team, the objective is to predict the discrete outcome of their upcoming match at Full Time (`FTR`):

- **Class 2:** Home Win
- **Class 1:** Draw
- **Class 0:** Away Win

**The "Draw" Problem and Class Imbalance:** A defining challenge of this dataset is the natural class imbalance. In the Premier League, Home Wins typically account for roughly 45% of outcomes, Away Wins ~30%, and Draws ~25%. Because Draws are statistically the rarest outcome and often occur as a byproduct of matched defensive strength rather than explicit tactical intent, machine learning models mathematically struggle to isolate a "Draw signal." Models naturally optimize for accuracy by aggressively predicting the majority classes and ignoring the Draw class entirely. A successful solution must implement specific loss functions and sampling techniques to prevent this statistical bias.

## 1.3 Project objectives

1. **Evaluate Sequence Modeling for Match Prediction**
   Determine whether modeling a team's recent match sequence using Neural Networks improves match outcome prediction compared with traditional static rolling averages over fixed windows (3, 5, 10 matches).
2. **Prevent Data Leakage in Feature Construction**
   Ensure all predictions are based purely on pre-match information by implementing chronological processing order (sorted by season and date) and feature computation from historical matches only. Make sure explicit season transition was handled (no mixing of future seasons) and opponent features merged using match_id and opponent, not future outcomes.
3. **Identify Key Predictive Feature Types**
   Determine which categories of engineered features contribute most to prediction accuracy and whether better feature engineering can contribute to the further improving the outcomes of classic ML models using rolling averages and modern Neural Networks using sequential information.
4. **Quantify Opponent Context Value**
   Measure how much extra predictive value comes from opponent-related features by comparing models that include opponent context against those that use only team-absolute statistics. This objective evaluates whether incorporating opponent-adjusted form, opponent strength ratings, and relative strength indicators such as Elo advantage improves match outcome prediction beyond what can be achieved by evaluating each team in isolation.

# 2.Data Engineering and Preprocessing

## 2.1 Original Dataset

The dataset used in this project was collected from a publicly available English Premier League results repository hosted on GitHub, covering seasons from 2000/01 to 2025/26. The original data is organized at the match level and contains variables describing match identity, match outcome, intermediate score state, and basic event statistics for both teams.

More specifically, the dataset includes the following features:

- **Date**: the calendar date on which the match was played.
- **HomeTeam**: the name of the home team.
- **AwayTeam**: the name of the away team.

- **FTHG (Full-Time Home Goals)**: the number of goals scored by the home team at the end of the match.
- **FTAG (Full-Time Away Goals)**: the number of goals scored by the away team at the end of the match.
- **FTR (Full-Time Result)**: the final categorical outcome of the match, typically encoded as Home Win, Draw, or Away Win.
- **HTHG (Half-Time Home Goals)**: the number of goals scored by the home team by half-time.
- **HTAG (Half-Time Away Goals)**: the number of goals scored by the away team by half-time.
- **HTR (Half-Time Result)**: the categorical result of the match at half-time.
- **Referee**: the official assigned to the match.
- **HS (Home Shots)**: the total number of shots attempted by the home team.
- **AS (Away Shots)**: the total number of shots attempted by the away team.
- **HST (Home Shots on Target)**: the number of home-team shots directed on target.
- **AST (Away Shots on Target)**: the number of away-team shots directed on target.
- **HF (Home Fouls)**: the number of fouls committed by the home team.
- **AF (Away Fouls)**: the number of fouls committed by the away team.
- **HC (Home Corners)**: the number of corner kicks awarded to the home team.
- **AC (Away Corners)**: the number of corner kicks awarded to the away team.
- **HY (Home Yellow Cards)**: the number of yellow cards received by the home team.
- **AY (Away Yellow Cards)**: the number of yellow cards received by the away team.
- **HR (Home Red Cards)**: the number of red cards received by the home team.
- **AR (Away Red Cards)**: the number of red cards received by the away team.

Taken together, these variables provide a compact but informative description of each fixture, capturing both the final outcome and several aspects of match performance. This makes the dataset suitable for constructing historical pre-match features, since past values of goals, shots, discipline, and other match statistics can be aggregated to represent recent team form and opponent context.

A previous version of the dataset also contained betting-odds variables. Such features are often highly informative in football prediction because they reflect aggregated market expectations and can indirectly encode information about team strength, injuries, and other contextual factors that may not be fully captured by standard match statistics alone. For this reason, betting odds could potentially improve predictive performance. However, they were intentionally excluded from this project in order to preserve a more academically meaningful setting, where the models are evaluated primarily on observable football data and engineered historical features rather than bookmaker-implied probabilities. This choice allows the analysis to focus more clearly on feature engineering, team form modeling, and opponent context.

| | Date | HomeTeam | AwayTeam | FTHG | FTAG | FTR | HTHG | HTAG | HTR | Referee | HS | AS | HST | AST | HF | AF | HC | AC | HY | AY | HR | AR |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 2025-08-15 | Liverpool | Bournemouth | 4 | 2 | H | 1 | 0 | H | A Taylor | 19 | 10 | 10 | 3 | 7 | 10 | 6 | 7 | 1 | 2 | 0 | 0 |
| 3 | 2025-08-16 | Aston Villa | Newcastle | 0 | 0 | D | 0 | 0 | D | C Pawson | 3 | 16 | 3 | 3 | 13 | 11 | 3 | 6 | 1 | 1 | 1 | 0 |
| 4 | 2025-08-16 | Brighton | Fulham | 1 | 1 | D | 0 | 0 | D | S Barrott | 10 | 7 | 4 | 2 | 16 | 15 | 4 | 3 | 3 | 3 | 0 | 0 |
| 5 | 2025-08-16 | Sunderland | West Ham | 3 | 0 | H | 0 | 0 | D | R Jones | 10 | 12 | 5 | 4 | 8 | 10 | 5 | 7 | 0 | 1 | 0 | 0 |
| 6 | 2025-08-16 | Tottenham | Burnley | 3 | 0 | H | 1 | 0 | H | M Oliver | 16 | 14 | 6 | 4 | 14 | 8 | 6 | 5 | 0 | 0 | 0 | 0 |
| 7 | 2025-08-16 | Wolves | Man City | 0 | 4 | A | 0 | 2 | A | J Gillett | 9 | 15 | 3 | 4 | 13 | 7 | 4 | 6 | 1 | 2 | 0 | 0 |
| 8 | 2025-08-17 | Chelsea | Crystal Palace | 0 | 0 | D | 0 | 0 | D | D England | 19 | 12 | 3 | 4 | 10 | 12 | 11 | 2 | 2 | 3 | 0 | 0 |

*Figure 2.1 : Original dataset columns as presented in the original github repo*

# 2.2 Preprocessing

## 2.2.1 Encoding categorical features

As part of preprocessing, nominal variables were transformed into numerical form so that they could be used by machine learning models. This mapping was applied to team names, referee names, season identifiers, and categorical match outcomes through fixed JSON dictionaries. The home-team and away-team mappings each contain 46 unique Premier League clubs, encoded consistently from 0 to 45, while the referee mapping contains 78 distinct referees appearing across the dataset. Season labels from 2000/01 through 2025/26 were also encoded sequentially from 0 to 25, preserving a compact numerical representation of the temporal span of the dataset. Finally, both the full-time result and half-time result variables were mapped identically as Away Win = 0, Draw = 1, and Home Win = 2, ensuring a consistent representation of outcome classes across the dataset.

## 2.2.2 Chronological Train–Validation–Test Splitting

To evaluate the models in a realistic forecasting setting, the dataset was divided chronologically into training, validation, and test sets rather than using a random split. The split chronologically is as follows train: from the start of the 2000/2001 season to the end of the 2016/2017 season, validation: from the start of the 2017/2018 season to the end of the 2020/2021 season, and test: from the start of the 2021/2022 season up until the end of the current season. This is important because football match prediction is inherently time dependent: the model should learn from past seasons and then be tested on later, unseen matches, exactly as it would be used in practice. A chronological split also prevents future matches from leaking into the training process, which would otherwise lead to overly optimistic and unrealistic performance estimates. This approach ensures that feature construction and model training always respect temporal order, meaning that a match can only use information from previous matches and not from

matches that occurred later in time. The same logic is especially important for rolling features and Elo ratings, since both must be computed strictly from historical matches and updated only after each match result becomes known.

## 2.3 ML feature engineering

### 2.3.1 Team-Centric Feature Engineering

Raw Premier League match data was first transformed into a team-centered representation, where each fixture is converted into two rows, one for the home team and one for the away team. In this format, the model sees the match from each team's perspective and learns to predict whether that team wins, draws, or loses. For every row, the feature set includes basic contextual information such as home or away status, opponent identity, pre-match Elo, rest days, and rolling statistics computed from the team's previous 3, 5, and 10 matches. These rolling features summarize recent form and performance history while remaining leakage-free, since they are based only on past matches.

### 2.3.2 Fixture-Level Feature Engineering

The data was also converted into a fixture-centered representation, where each match remains a single row. Here, the historical features of the home team and away team are combined into one observation, and additional difference-based features are created to capture the relative strength of the two sides before kickoff. This representation includes pre-match Elo values as well as home-minus-away comparisons of key rolling statistics, allowing the model to learn directly from matchup differences rather than from each team in isolation. The target in this setting is the actual match result: Home Win, Draw, or Away Win.

### 2.3.3 Overview of the features used for ML algorithms

- **Team and Match Context**

The feature set begins with basic identifiers and match context, including the fixture ID, date, season, split, team, opponent, and the team-perspective target label. Additional calendar variables such as month, day of week, and weekend indicator capture scheduling effects, while season progress and the early-season flag help model how team information evolves over the course of a season. Rest-related variables, such as days since the previous match and match density, are included to reflect fatigue and fixture congestion, both of which can influence

performance. Note that this context is difficult to quantify without specialized performance and fitness metrics, only available to the team staff and not provided through this dataset.

- **Strength and Carryover**

To represent longer-term team strength, the features include pre-match Elo ratings (to be discussed in the next paragraph) for both the team and the opponent, along with their difference. These ratings are updated only after each match, so they represent leakage-free strength estimates available before kickoff. We also include previous-season summary statistics, such as points per game and goals for and against per game, to capture carryover between seasons and provide a more stable estimate of team quality, especially at the start of a new campaign. The promoted-team flag is used to mark cases where a club is new to the league or has no recent top-flight history, which helps the model treat cold-start situations differently.

- **Recent Form**

Recent form is represented through rolling statistics over the last 3, 5, and 10 matches. For each window, we compute average form, win rate, goals scored, goals conceded, goal variance, goal balance, and a simple momentum measure that compares short-term and longer-term form. These features summarize how well a team has been performing recently and are intended to capture short-run trends that may not be visible in season-wide averages. The same set of features is computed for the opponent as well, so the model can compare both sides of the fixture rather than evaluating only one team in isolation.

### 2.3.4 Elo Rating Features

The Elo ratings system, developed by Dr. Arpad Elo, was originally used in chess but has since been applied in a wide range of sports. In 1997 Bob Runyan adapted the Elo rating system to international football and maintained the World Football Elo Ratings website, recording teams' dynamics worldwide based on the Elo system . The system was adapted to football by adding a weighting for the kind of match, an adjustment for the home team advantage, and an adjustment for goal difference in the match result

In the Elo system, teams typically begin with a rating of 1500. The method works in two stages: first, it estimates a team's chance of winning a match (E-step) , and then it adjusts the team's rating after the result is known (U-step).The basic idea is that a team gains fewer points when it beats a much weaker opponent and more points when it beats a stronger one, while the opposite happens when it loses.

- **E-step**

Without loss of generality the probability that team $i$ beats team $j$ is estimated by:

$$p_{i,j}(t) = \frac{1}{1+10^{\frac{-(R_i(t)-R_j(t))}{400}}}$$

Where $R_i(t)$ represents the Elo rating of team i at the given time t.

Since these are probabilities, they add to one, i.e., $p_{i,j}(t) + p_{j,i}(t) = 1$.

- **U-step**

Without loss of generality the Elo ratings after the match is updated for each team according to the following formula :

$$R_i(t + 1) = R_i(t) + K[A_i(t) - p_{i,j}(t)]$$

Note that $A_i(t)$ takes the values 1 for i's team win, 0 for loss and 0.5 for draw.

After each match, Elo ratings are updated by comparing the actual result with the result that was expected from the pre-match ratings. If a team performs better than expected, its rating rises; if it performs worse, its rating falls. The size of the change is controlled by the K-factor, which determines how sensitive the ratings are. A large K makes ratings react quickly but can make them unstable, while a small K makes them smoother but slower to adapt. Some versions also make K depend on goal difference so that bigger wins or losses produce larger rating changes.

### 2.3.5 Leakage prevention

Leakage prevention is built into the pipeline to ensure that every model only sees information available before kickoff. Leakage is prevented throughout the pipeline by ensuring that every feature is computed strictly from past information. In process_split(), the code first calls get_team_features() to build the input features from stored history, and only afterward calls update_after_match() to refresh the team state with the current match result. This means the current fixture never influences its own feature vector. In addition, select_final_columns() keeps only pre-match and historical variables, while split_for_ml.py removes the target and metadata columns before constructing the final ML matrices, so the model never sees direct outcome columns such as goals, wins, or full-time result.

## 2.4 Sequence Dataset Construction

While the classical machine learning models developed in this project operate on tabular feature representations, recurrent neural architectures require sequential inputs that preserve the temporal ordering of past matches. Therefore, a dedicated sequence generation pipeline was designed to transform historical football records into fixed-length time series samples suitable for deep learning models. Note that every sequence is built exclusively from matches that occurred prior to the target fixture.

### 2.4.1 Home and Away Historical Sequences

For each target match, the historical performance of both competing teams is extracted. Given a sequence length N, the pipeline retrieves the previous N matches of the home team and the previous N matches of the away team.

For  example, consider the fixture:

Liverpool vs Arsenal

If N = 10, the model receives:

- Liverpool's previous 10 matches
- Arsenal's previous 10 matches

And is asked to predict the outcome of the upcoming fixture.

The target match itself is never included in the sequence, ensuring that no future information is available during training or evaluation.

### 2.4.2 Per Match Feature Representation

Each historical match is represented using a set of pre match historical performance indicators. The final sequence representation  includes the following features of each team:

- Home/Away indicator
- Points obtained
- Goals scored
- Goals conceded
- Goal difference
- Shots for
- Shots against
- Shots on target for
- Shots on target against
- Corners for
- Corners against
- Yellow cards for
- Yellow cards against
- Red cards for
- Red cards against
- Rest days

- Elo rating

### 2.4.3 Sequence Tensor Construction

The Project primarily employs a home-away sequence representation. For every timestep, the feature vectors of the home and away teams are concatenated into a single representation.

After the introduction of Elo ratings, each team contributes 17 features per timestep. Therefore:

- Home away features: 17
- Away team features: 17
- Total timestep features: 34

For a sequence length of 10 matches, the resulting neural network input has the form:

$X \in R^{\{10 \times 34\}}$ ,

More generally, the sequence tensor is represented as:

$X \in R^{\{B \times N \times F\}}$

where :

- B denotes the batch size,
- N denotes the sequence length,
- F denotes the number of features per timestep.

In the experiments conducted, sequence lengths of 5, 10 , 50 and 100 were evaluated.

### 2.4.4 Leakage Free Temporal Alignment

A critical design requirement of the sequence pipeline is strict temporal alignment. Every timestep within a sequence corresponds to a match that occurred before the target fixture.

Furthermore:

- rolling statistics are computed using shifted historical windows,
- Elo ratings are recorded before the target match is played,

- validation and test samples are generated chronologically,
- future matches never contribute information to earlier observations.

As a result, the generated sequence datasets remain fully consistent with the real world football forecasting scenario and preserve the leakage free design principles adopted throughout this project.

### 2.4.5 Raw and Enhanced Sequence Variants

Two sequence representations were explored during experimentation.

The first representation, referred to as the raw sequence dataset, contains only historical match-level statistics and Elo ratings. This representation served as the primary input for the Scratch RNN, GRU, and LSTM models.

The second representation augments each timestep with additional rolling historical features derived from the feature engineering pipeline. Although this richer representation substantially increases the dimensionality of the input tensors, experimental results showed only marginal improvements and, in some cases, increased overfitting. Consequently, the raw sequence representation was retained as the default sequential dataset used throughout the final experiments.

# 3. Methodology and Architectures

## 3.1 Classical Machine Learning Baselines

To establish a rigorous performance floor before exploring complex sequential neural networks, a suite of classical machine learning algorithms was evaluated. These algorithms operate on the static, tabular feature representations generated by the feature engineering pipeline. The machine learning pipeline utilized standard `scikit-learn` preprocessing steps, including imputation for missing values, feature scaling for linear models, and one-hot encoding for categorical variables (e.g., season identifiers and team names).

- **Dummy Classifier (Stratified):** Serving as a fundamental sanity check, this baseline generates predictions by randomly sampling from the training set's class distribution, rather than simply predicting the majority class. Based on 1,890 test samples, it achieved an accuracy of 34.13%, a Macro F1 score of 31.91%, and a Weighted F1 of 34.04%. These metrics serve as the effective performance floor; any trained model that

fails to significantly outperform these values lacks predictive utility, as it performs no better than chance-based guessing aligned with historical class frequencies.

- **Logistic Regression:** A linear classifier that learns weights for each historical feature to produce class probabilities. Despite its mathematical simplicity, Logistic Regression proved highly effective when paired with heavily engineered features. Notably, it achieved the best classical Macro F1 score (42.37% at the fixture level) because its probabilistic nature responded well to class weighting, allowing it to better identify the highly ambiguous "Draw" class.
- **Random Forest:** An ensemble of decision trees where each tree learns specific feature thresholds and the forest averages their votes. This approach naturally handles non-linear relationships and complex interactions between team statistics. It achieved the highest fixture-level accuracy prior to the introduction of Elo ratings (52.11%).
- **XGBoost:** An optimized implementation of gradient boosting that builds decision trees sequentially, with each new tree attempting to correct the residual errors of previous iterations. XGBoost ultimately achieved the absolute highest accuracy among all classical models (52.43%) when augmented with Elo ratings.

## 3.2 Tabular Data Representations: Team-Perspective vs. Fixture-Level

For the classical machine learning models, the historical match data was structured into two distinct tabular representations to evaluate how the algorithms best learn football dynamics:

- **Team-Perspective Modeling:** In this format, every target fixture is split into two separate observations—one from the perspective of the home team, and one from the perspective of the away team. The target variable is simplified to a team-centric outcome: Win (0), Draw (1), or Loss (2). This approach doubles the dataset size (yielding 12,614 training rows and 103 features) and forces the model to learn what statistical thresholds make a generic team win or lose, evaluating each side somewhat in isolation.
- **Fixture-Level Modeling:** This format retains the natural structure of the sport, representing each match as a single row. The feature space explicitly combines the pre-match history of the home team (`home_*`), the pre-match history of the away team (`away_*`), and direct mathematical differentials between the two (`diff_*`). The target remains the standard Home Win (0), Draw (1), or Away Win (2). This representation (yielding 6,307 training rows and 151 features) aligns perfectly with real-world forecasting by framing the match as a direct clash between two competing forms. Experimental results demonstrated that this fixture-level representation yielded stronger overall predictive stability.

## 3.3 The Impact of Elo Ratings on Tabular Models

A key experimental phase involved integrating Elo ratings into the classical machine learning pipeline to measure their standalone predictive value. Because Elo ratings are updated strictly post-match and carried forward, they provide a mathematically rigorous, leakage-free estimate of long-term team strength that bridges the gap between historical reputation and current form.

In the fixture-level representation, introducing Elo added three specific variables to the dataset: `home_elo`, `away_elo`, and `elo_diff`. The inclusion of these features demonstrated a consistent, positive impact across the models. Most notably, fixture-level XGBoost saw its accuracy improve from 51.52% to 52.43%. Random Forest and Logistic Regression also saw stabilization or slight improvements in their respective metrics. The results confirmed that Elo ratings provide a highly interpretable, low-dimensional signal that enhances traditional ML accuracy, successfully answering one of the core research questions before transitioning into the computationally expensive sequential architectures described in the following sections.

## 3.4 Convolutional Neural Networks (CNN)

### 3.4.1. CNN Architecture

The CNN-based models were designed as lightweight residual architectures for sequence classification, with an input tensor of shape (N,F,L), where N is the batch size, F the number of engineered features, and L the sequence length. Each convolutional block applies two same-padded convolutions so that the temporal resolution is preserved within the block, followed by batch normalization, ReLU activation, and dropout; a residual shortcut is added before the final activation to improve optimization stability and gradient flow. In the 1D variant, the shortcut is an identity mapping when the channel dimension is unchanged, and a 1×1 convolution otherwise.

The 1D CNN treats the features as channels and the sequence axis as time. Starting from (N,F,L), it passes through three residual 1D convolutional blocks with intermediate max-pooling, so the temporal dimension is gradually reduced while the channel dimension increases; the final feature map is then compressed with adaptive average pooling and adaptive max pooling to (N,C,1), where C is the last channel size. After squeezing and concatenating the pooled descriptors, the classifier receives a vector of size 2C and outputs logits of shape (N,3) for the three match classes.

The 2D CNN first reshapes the same sequence into a 2D tensor of shape (N,1,L,F), so the model can learn local patterns jointly across time and feature dimensions. It then applies residual 2D convolutional blocks with max-pooling only along the feature axis, which keeps the temporal ordering intact while progressively abstracting feature interactions; after the

convolutional stack, adaptive average pooling and adaptive max pooling reduce the representation to (N,C,1,1). These pooled outputs are concatenated into a 2C-dimensional vector and mapped by the classifier to (N,3).

The hybrid CNN combines both views of the same input. The 1D branch receives (N,F,L) and focuses on temporal dynamics, while the 2D branch receives (N,1,L,F) and captures local time-feature interactions; each branch ends with adaptive average and max pooling, producing vectors of size 2C1D  and 2C2D, respectively. These two representations are concatenated into a fused vector of size 2C1D+2C2D, which is then passed through a compact fully connected classifier to produce the final logits (N,3). This setup preserves the original design while making the tensor flow and dimensionality changes explicit

*Code:  “Simplified architecture of the 1D CNN model used for match outcome prediction. The network processes inputs of shape(N,F,L), where N is the batch size, F the number of features, and L the sequence length. It stacks residual convolutional blocks with max-pooling, then uses global average and max pooling before a fully connected classifier outputs three-class logits.“*

```
class ConvBlock1D(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, dropout=0.2):
        super().__init__()
        padding = kernel_size // 2
        self.conv = nn.Sequential(
            nn.Conv1d(in_channels, out_channels, kernel_size=kernel_size, padding=padding, bias=False),
            nn.BatchNorm1d(out_channels),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Conv1d(out_channels, out_channels, kernel_size=kernel_size, padding=padding, bias=False),
            nn.BatchNorm1d(out_channels),
        )
        self.shortcut = nn.Identity() if in_channels == out_channels else nn.Conv1d(in_channels, out_channels,
kernel_size=1, bias=False)
        self.activation = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.activation(self.conv(x) + self.shortcut(x))


class Conv1DMatchPredictor(nn.Module):
    def __init__(self, input_channels, num_classes=3, channels=None, dropout=0.3, classifier_hidden=128):
        super().__init__()
        channels = channels or [128, 128, 256]
        blocks, in_channels = [], input_channels
        for out_channels in channels:
            blocks += [ConvBlock1D(in_channels, out_channels, dropout=dropout), nn.MaxPool1d(2, 2,
ceil_mode=True)]
            in_channels = out_channels
        self.features = nn.Sequential(*blocks)
        self.global_avg_pool = nn.AdaptiveAvgPool1d(1)
        self.global_max_pool = nn.AdaptiveMaxPool1d(1)
        self.classifier = nn.Sequential(
            nn.Linear(in_channels * 2, classifier_hidden),
            nn.BatchNorm1d(classifier_hidden),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(classifier_hidden, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.cat([self.global_avg_pool(x).squeeze(-1), self.global_max_pool(x).squeeze(-1)], dim=1)
        return self.classifier(x)
```

### 3.4.2. CNN hyper-parameter sweep

The CNN hyperparameter search was conducted in three successive stages to isolate the effect of each training choice while keeping the architecture fixed. First, the script swept the learning rate over {1×10^(-4) ,3×10^(-4) ,1×10^(-3)}, because learning rate has the strongest effect on convergence speed and training stability. Second, it swept the dropout rate over {0.2,0.3,0.5}, since dropout controls regularization strength and helps reduce overfitting. Third, it swept the

batch size over {32,64,128}, which affects gradient noise, optimization stability, and memory usage.

The selection criterion was validation Macro F1, followed by validation accuracy and then validation loss, so the search favored balanced class performance rather than accuracy alone. The model was trained with AdamW and fixed weight decay {1*10^(-3)}, which acts as an additional regularizer by discouraging large weights. The loss function was standard cross-entropy, and the code did not use focal loss in this sweep; if focal loss were used, it would mainly reweight difficult or minority-class examples to address imbalance, but this particular experiment relied on class-balanced evaluation metrics instead. Early stopping with patience 8 and gradient clipping at norm 5.0 were also used to improve training stability.

### 3.4.3. CNN over hand-crafted ML features turned into sequences

The first CNN attempt used the same engineered rolling features that powered the classical ML pipeline, but arranged as sequential input so the CNN could process them over time. In practice, this version performed worse than the CNNs trained on the raw sequence representation, which suggests that the engineered rolling averages were not helping the convolutional model extract cleaner temporal patterns. A plausible explanation is that these features introduce extra redundancy and noise: the CNN already sees short match-history windows, so repeatedly injecting rolling aggregates such as multi-window averages may blur the signal rather than sharpen it.

After that, the pipeline was simplified by removing most of the rolling averages and keeping only average_5. That change was meant to reduce feature clutter and test whether a more compact summary of recent form would improve generalization, but the result still suggested that the raw-sequence version was the stronger representation. This supports the interpretation that, for the CNN setting, the model benefits more from learning temporal patterns directly from match histories than from being fed handcrafted summary statistics at every timestep.

### 3.4.4. CNN over raw sequences

The second attempt, based on the raw sequence representation, worked better because it let the CNN learn its own temporal features directly from the match history instead of relying on handcrafted summaries. In this setting, the model could exploit the full ordering of events, capture short- and mid-range dependencies through convolution, and build hierarchical representations without being constrained by precomputed rolling aggregates. That often helps when the engineered features are noisy, redundant, or too smoothed, because the network receives a cleaner signal and can decide which patterns matter during training.

In practical terms, the raw input preserved more of the original variation in the sequence, while the rolling features compressed the history into overlapping statistics that may have hidden useful local changes. The CNN therefore had a better chance of learning discriminative motifs from the raw windows than from a representation that already mixed multiple time scales into each feature vector. This explains why the raw-sequence version was the stronger baseline and why it is reasonable to treat it as the preferred input formulation for the CNN experiments.

## 3.5 Recurrent Neural Network Models

Team performance evolves over time, and recent results often carry different predictive values than older ones. To capture these temporal dependencies, recurrent neural network architectures were investigated.

Unlike classical machine learning models, recurrent architectures process historical match sequences in chronological order. At each timestep, the network receives information describing a previous match and updates an internal hidden representation that summarizes the team's recent form. After processing the complete sequence, the final hidden representation is used to predict the outcome of the upcoming fixture.

The recurrent models developed in this project operate on the sequence datasets described in Section 2.4, where each sample consists of historical match information for both the home and away teams. Several sequence lengths were explored throughout the experiments, including 5, 10, 50, and 100 previous matches

### 3.5.1 Recurrent Neural Network (RNN)

The first recurrent model evaluated was a vanilla Recurrent Neural Network. At each timestep, the model receives the feature vector of a historical match and combines it with the previous hidden state. The hidden state acts as a compact memory of the sequence processed so far.

For each timestep, the RNN updates its hidden state using the current input and the previous hidden state. After the final timestep, the last hidden state is passed to a classification layer that outputs three logits corresponding to Home Win, Draw, and Away Win.

This architecture served as the main recurrent baseline. It was tested with different sequence lengths, including 5, 10, 50, and 100 previous matches, as well as different hidden dimensions and class-weighting strategies.

### 3.5.2 Gated Recurrent Unit (GRU)

A Gated Recurrent Unit was implemented to investigate whether a gated recurrent architecture could improve performance over the vanilla RNN. GRUs are designed to handle longer temporal

dependencies more effectively by controlling how much past information is retained or updated at each timestep.

The implemented GRU uses update and reset gates. The update gate controls how much of the previous hidden state is preserved, while the reset gate controls how much past information is used when computing the candidate hidden state. These mechanisms allow the model to selectively retain or discard information from earlier matches in the sequence.

*Code 1: Manual implementation of the GRU update equations used in the proposed football prediction framework.*

```
  def forward(self, x_t: torch.Tensor, h_prev: torch.Tensor) ->
torch.Tensor:
      z_t = torch.sigmoid(
          self.input_to_update(x_t)
          + self.hidden_to_update(h_prev)
          + self.update_bias
      )
      r_t = torch.sigmoid(
          self.input_to_reset(x_t)
          + self.hidden_to_reset(h_prev)
          + self.reset_bias
      )
      h_candidate = torch.tanh(
          self.input_to_candidate(x_t)
          + self.hidden_to_candidate(r_t * h_prev)
          + self.candidate_bias
      )
      return (1.0 - z_t) * h_prev + z_t * h_candidate
```

- *Update **gate z_t**: controls how much of the new candidate state replaces the previous hidden state.*
- *Reset **gate r_t**: controls how much previous memory is used when computing the candidate hidden state.*
- *Candidate **state h_candidate**: proposes a new hidden representation from the current input and reset-filtered previous state.*
- ***Final hidden update**: interpolates between old memory h_prev and new memory h_candidate.*

As with the RNN, the GRU was evaluated on home-away historical sequences and compared under different sequence lengths, hidden dimensions, class-weighting schemes, and regularization settings.

### 3.5.3 Long Short-Term Memory Network

Long Short-Term Memory networks were also evaluated as part of the deep sequential learning experiments. LSTMs extend the recurrent framework by using a memory cell and multiple gates to control information flow over time. This design is intended to reduce the vanishing gradient problem and improve the ability to learn longer-term dependencies.

In this project, the LSTM models were trained on the same sequence datasets used by the RNN and GRU experiments. Their role was to provide an additional comparison point for assessing whether more complex recurrent memory mechanisms improve football match outcome prediction.

Code 2: LSTM architecture for encoding historical football match sequences and predicting match outcome classes."

```
self.encoder = nn.LSTM(
    input_size=input_size,
    hidden_size=hidden_size,
    num_layers=num_layers,
    batch_first=True,
    dropout=lstm_dropout,
    bidirectional=bidirectional,
)
classifier_hidden_size = classifier_hidden_size or hidden_size
classifier_input_size = hidden_size * self.num_directions
self.classifier = nn.Sequential(
    nn.Dropout(dropout),
    nn.Linear(classifier_input_size, classifier_hidden_size),
    nn.ReLU(inplace=True),
    nn.Dropout(dropout),
    nn.Linear(classifier_hidden_size, num_classes),
)
def forward(self, sequences: torch.Tensor) -> torch.Tensor:
    _, (hidden, _) = self.encoder(sequences)
    if self.bidirectional:
        # Concatenate last forward and backward hidden states
        final_hidden = torch.cat([hidden[-2], hidden[-1]], dim=1)
    else:
        final_hidden = hidden[-1]
    return self.classifier(final_hidden)
```

- ***Input shape****: expected as (batch_size, sequence_length, input_size) because batch_first=True*
- ***LSTM layer****: nn.LSTM() encodes the full match history sequence.*
- *Final Hidden representation: uses hidden[-1] for normal LSTM, or concatenates final forward/backward states for bidirectional LSTM*
- *Classifier output: final hidden vector is passed through a feed-forward classifier to produce logits of shape (batch_size, num_classes), where num_classes = 3*

The LSTM models were trained on the same sequence datasets used by the Scratch RNN and Scratch GRU experiments, allowing a direct comparison between manually implemented recurrent architectures and a standard library-based recurrent model.

## 3.6 Hybrid Deep Learning Models

Deep learning architectures often specialize in different aspects of sequential data. Convolutional Neural Networks (CNNs) are effective at extracting local patterns and short-term relationships, while recurrent architectures such as LSTMs are designed to model temporal dependencies across longer sequences. To investigate whether combining these complementary capabilities could improve football match outcome prediction, several hybrid architectures were evaluated.

The hybrid models developed in this project combine convolutional feature extraction with sequential modeling components. Their objective is to identify informative local patterns within historical match sequences while simultaneously preserving longer-term temporal information. These architectures were evaluated using the same leakage-free sequence datasets described in Section 2.4 and were compared against both classical machine learning approaches and standalone recurrent models

### 3.6.1 CNN-LSTM Architecture

The first hybrid models architecture combines one-dimensional convolutional layers with a Long Short-Term Memory network.

The intuition behind this approach is that the convolutional component can automatically detect short-term patterns within the sequence, such as recent changes in team form, scoring behaviour, or momentum trends. These extracted representations are subsequently passed to an LSTM layer, which models the temporal evolution of these patterns across the entire historical sequence.

The input sequence is first processed by a series of one-dimensional convolutional filters operating along the temporal dimension. The resulting feature maps are then fed into an LSTM encoder, which produces a compact hidden representation summarizing the sequence. Finally, a fully connected classification network predicts one of the three possible match outcomes: Home Win, Draw, or Away Win

*Code: "CNN-LSTM hybrid architecture where Conv1D layers extract local temporal patterns from team-history sequences, and an LSTM encoder summarizes them for three-class match outcome prediction."*

```
  blocks: list[nn.Module] = []
  in_channels = input_features
  for out_channels in conv_channels:
      blocks.append(
          Conv1DBlock(
              in_channels=in_channels,
              out_channels=out_channels,
              kernel_size=kernel_size,
              dropout=dropout,
          )
     )
      in_channels = out_channels
  self.feature_extractor = nn.Sequential(*blocks)
  self.lstm = nn.LSTM(
      input_size=conv_channels[-1],
      hidden_size=lstm_hidden_size,
      num_layers=lstm_layers,
      batch_first=True,
      bidirectional=bidirectional,
      dropout=dropout if lstm_layers > 1 else 0.0,
  )

  classifier_input_features = lstm_hidden_size * (2 if bidirectional else
1)
  self.classifier = nn.Sequential(
      nn.Linear(classifier_input_features, classifier_input_features),
      nn.BatchNorm1d(classifier_input_features),
      nn.ReLU(inplace=True),
      nn.Dropout(dropout),
      nn.Linear(classifier_input_features, num_classes),
  )
  def forward(self, x: torch.Tensor) -> torch.Tensor:
      x = x.transpose(1, 2)
      x = self.feature_extractor(x)
      x = x.transpose(1, 2)

      lstm_output, (h_n, _) = self.lstm(x)

      if self.bidirectional:
          hidden = torch.cat([h_n[-2], h_n[-1]], dim=1)
      else:
          hidden = h_n[-1]

      logits = self.classifier(hidden)
      return logits
```

- Input shape is (batch_size, sequence_length, num_features).
- x.transpose(1, 2) changes it to Conv1D format: (batch_size, num_features, sequence_length).
- self.feature_extractor applies Conv1D blocks to detect short-term local form patterns.
- The tensor is transposed back to (batch_size, sequence_length, conv_features) for the LSTM.
- nn.LSTM models temporal evolution across the sequence.
- The final hidden state h_n[-1] becomes the compact match representation.
- self.classifier outputs 3 logits for HomeWin, Draw, and AwayWin.

### 3.6.2 CNN-LSTM Variants

In addition to the baseline CNN-LSTM architecture, several enhanced variants were investigated.

These models retain the same general CNN-to-LSTM processing pipeline but modify how information is aggregated after the recurrent stage. Instead of relying exclusively on the final hidden state of the LSTM, the entire sequence of LSTM outputs can be summarized using pooling operations.

More specifically, global average pooling and global max pooling were applied to the sequence of hidden states generated by the LSTM. The resulting pooled representations were concatenated and passed to the final classification layer.

This design allows the model to utilize information distributed across the entire sequence rather than relying solely on the final recurrent state.

Code: “CNN-LSTM variant using global average pooling and max pooling over all LSTM hidden states before final match outcome classification.”

```
x = x.transpose(1, 2).contiguous()
x = self.feature_extractor(x)
x = x.transpose(1, 2).contiguous()
lstm_output, (h_n, _) = self.lstm(x)
if self.use_pooling_summary:
    mean_pool = lstm_output.mean(dim=1)
    max_pool, _ = lstm_output.max(dim=1)
    features = torch.cat([mean_pool, max_pool], dim=1)
else:
    if self.bidirectional:
        features = torch.cat([h_n[-2], h_n[-1]], dim=1)
    else:
```

```
        features = h_n[-1]

logits = self.classifier(features)
return logits
```

- The input is first transposed so Conv1D can process the temporal sequence.
- **self.feature_extractor(x)** applies the CNN feature extractor.
- The output is transposed back into LSTM format.
- **lstm_output** contains hidden representations for every timestep.
- **mean_pool** captures the average behaviour across the whole sequence.
- **max_pool** captures the strongest detected temporal signal.
- These pooled vectors are concatenated and passed to the classifier.
- **logits** are the final scores for HomeWin, Draw, and AwayWin.

### 3.6.3 Home Away Hybrid Models

A further family of hybrid architectures was designed to explicitly preserve the distinction between the historical trajectories of the home and away teams.

Instead of concatenating both teams into a single sequence representation at the input stage, the model processes the historical records of each team separately. Independent feature extraction pathways are applied to the home-team sequence and the away-team sequence before their learned representations are combined.

This design is motivated by the observation that home and away performance often exhibit different statistical characteristics. By allowing the model to learn separate representations for each team, it becomes possible to capture asymmetric behaviour that may be lost when both histories are merged too early.

After the independent encoders produce their respective representations, the resulting feature vectors are concatenated and passed to a final classification network that predicts the match outcome.

The home-away hybrid approach therefore combines the benefits of specialized team-specific representations with the predictive power of a shared classification layer, enabling a richer modeling of football match interactions.

*Code:"Home-away hybrid CNN architecture that processes home and away team histories through separate branches before fusing their representations for match outcome classification."*

```
  self.home_branch = self._make_conv1d_branch(self.team_channels, conv1d_filters,
dropout)
  self.away_branch = self._make_conv1d_branch(self.team_channels, conv1d_filters,
dropout)

  self.cross_branch = self._make_conv2d_branch(1, conv2d_filters, dropout)

  self.home_pool = nn.AdaptiveAvgPool1d(1)
  self.away_pool = nn.AdaptiveAvgPool1d(1)
  self.cross_pool = nn.AdaptiveAvgPool2d((1, 1))

  fusion_dim = conv1d_filters[-1] * 2 + conv1d_filters[-1] + conv2d_filters[-1]
  self.classifier = nn.Sequential(
     nn.Linear(fusion_dim, hidden_size),
     nn.ReLU(inplace=True),
     nn.Dropout(dropout),
     nn.Linear(hidden_size, num_classes),
  )

  home_x = x[:, :, : self.team_channels].transpose(1, 2)
  away_x = x[:, :, self.team_channels :].transpose(1, 2)

  home_x = self.home_branch(home_x)
  away_x = self.away_branch(away_x)

  home_x = self.home_pool(home_x).squeeze(-1)
  away_x = self.away_pool(away_x).squeeze(-1)

  diff_x = torch.abs(home_x - away_x)

  cross_x = x.unsqueeze(1)
  cross_x = self.cross_branch(cross_x)
  cross_x = self.cross_pool(cross_x).view(x.size(0), -1)

  fused = torch.cat([home_x, away_x, diff_x, cross_x], dim=1)
  logits = self.classifier(fused)
  return logits
```

- The input has shape (batch_size, sequence_length, num_features).
- The feature dimension is split into two halves:

  -first half: home-team historical features

-second half: away-team historical features

- **home_branch** learns a representation of the home team's recent history.
- **away_branch** learns a separate representation of the away team's recent history.
- **home_pool** and **away_pool** compress each team's sequence into one vector.
- diff_x = torch.abs(home_x - away_x) explicitly models the difference between the two teams.
- **cross_branch** processes the full combined input as an interaction branch.
- All representations are concatenated in fused.
- The classifier outputs 3 logits: HomeWin, Draw, and AwayWin.

## 3.7 Handling Class Imbalance

Football match prediction is naturally affected by class imbalance. In the Premier League, home wins occur more frequently than draws, while draws are usually the hardest class to predict. This creates a bias during training, because a model can achieve reasonable accuracy by mostly predicting the more common outcomes and ignoring draws.

In this project, this issue was important because accuracy alone was not enough to evaluate model quality. A model that predicts many home wins correctly may achieve acceptable accuracy, but still perform poorly on the draw class. For this reason, Macro F1 was used alongside accuracy, since it evaluates all three classes more equally.

### 3.7.1 Weighted Loss Functions

To reduce the effect of class imbalance, weighted loss functions were used in several neural network experiments. In standard cross-entropy loss, every training example contributes equally to the loss. With class weighting, errors on underrepresented classes receive larger penalties.

This means that mistakes on draws can be made more costly during training, encouraging the model to pay more attention to the draw class.

The experiments showed a clear trade-off. Weighted models often achieved lower overall accuracy, but improved Macro F1. This indicates that they became more balanced across the three classes, even if they made slightly fewer total correct predictions.

*Code 3: Class-weighted cross-entropy loss used to reduce class imbalance, with optional additional weighting for the draw class.*

```
  def make_class_weights(y_train, class_weighting, draw_weight):
      if class_weighting == "none":
          return None

      counts = class_counts(y_train).astype(np.float32)
      total = float(counts.sum())
      weights = total / (len(counts) * counts)

      if draw_weight is not None:
          weights[1] *= float(draw_weight)  # class 1 = Draw

      return torch.tensor(weights, dtype=torch.float32)

  counts = class_counts(train_y)
  weights = make_class_weights(train_y, args.class_weighting,
args.draw_weight)
  criterion = nn.CrossEntropyLoss(
      weight=weights.to(device) if weights is not None else None
  )
```

### 3.7.2 Draw Prediction Challenges

The draw class was consistently the most difficult outcome to predict. This is expected in football, because draws often occur in matches where the teams are closely matched and where small random events can change the final result.

Unlike home wins or away wins, draws do not always correspond to a strong positive signal. They may result from defensive balance, missed chances, low-scoring variance, or simply randomness. As a result, models often confuse draws with narrow home or away wins.

This explains why some models achieved stronger accuracy but weaker Macro F1. They performed well on the majority classes but failed to identify draws reliably. Weighted loss helped reduce this issue, but it did not fully solve the draw prediction problem

## 3.8 Training Procedure

To ensure a fair comparison between the different machine learning and deep learning approaches, a consistent training and evaluation procedure was followed throughout the project. Model development was performed using the training set, hyperparameter tuning was conducted on the validation set, and all final performance metrics were reported on the held-out test set.

The chronological train-validation-test split described in Section 2 was maintained throughout all experiments to preserve the temporal nature of football forecasting and prevent information leakage.

### 3.8.1 Hyperparameter Selection

Several hyperparameters were investigated during the development of the proposed models.

For the classical machine learning algorithms, parameters such as tree depth, number of estimators, learning rate, and regularization strength were tuned using validation performance.

For the deep learning models, experimentation focused primarily on:

- Sequence length (5, 10, 50, and 100 matches)
- Hidden state dimensionality
- Number of recurrent layers
- Dropout rate
- Learning rate
- Batch size
- Class weighting strategies

Multiple configurations were evaluated for each architecture in order to identify suitable trade-offs between predictive performance and model complexity. Hyperparameter decisions were guided primarily by validation set performance rather than training performance to reduce the risk of overfitting.

### 3.8.2 Early Stopping

To improve generalization and reduce unnecessary training time, early stopping was employed during the training of the deep learning models.

After each epoch, model performance was evaluated on the validation set. Training was terminated when the validation metric failed to improve for a predefined number of consecutive epochs. The best-performing model checkpoint observed during training was retained for final evaluation on the test set.

This strategy proved particularly important for recurrent architectures trained on longer sequences, where overfitting frequently emerged after only a few epochs. Early stopping helped prevent the models from continuing to optimize the training set while simultaneously degrading validation performance.

By selecting the checkpoint corresponding to the best validation performance, the resulting models achieved more stable and reproducible generalization behaviour.

*Code: "Early stopping mechanism used during recurrent model training, where the best checkpoint is selected according to validation macro F1 and restored before final evaluation."*

```
if float(val_metrics["macro_f1"]) > best_val_macro_f1:
    best_val_macro_f1 = float(val_metrics["macro_f1"])
    best_epoch = epoch
    epochs_without_improvement = 0
    best_state = {
        key: value.detach().cpu().clone()
        for key, value in model.state_dict().items()
    }
else:
    epochs_without_improvement += 1
    if early_stopping and epochs_without_improvement >= patience:
        stopped_epoch = epoch
        break
if best_state is not None:
    model.load_state_dict(best_state)
return model, history, best_epoch, stopped_epoch
```

- **val_metrics["macro_f1"]** is checked after each epoch.
- If validation macro F1 improves, the model saves a copy of the current weights in best_state.
- **epochs_without_improvement** resets to zero after improvement.
- If validation performance does not improve, the counter increases.
- When the counter reaches patience, training stops early.
- After training, **model.load_state_dict(best_state)** restores the best validation checkpoint, not necessarily the final epoch.

## 3.8.3 Model Selection Strategy

Model selection was performed using validation-set performance. The validation set was used to compare alternative architectures, sequence representations, hyperparameter configurations, and class-weighting schemes.

Two complementary evaluation metrics were considered during model selection:

- **Accuracy**, which measures overall prediction correctness
- **Macro F1 Score**, which evaluates performance equally across Home Win, Draw, and Away Win outcomes

The use of both metrics was motivated by the class imbalance present in football match outcomes. While accuracy rewards overall correctness, Macro F1 provides a more balanced assessment of performance across all classes, particularly the more difficult draw category.

After model selection was completed, the chosen configuration was evaluated once on the test set. The test set was never used for hyperparameter tuning or architecture selection, ensuring an unbiased estimate of final model performance.

# 4. Evaluation Strategy

## 4.1 Evaluation Protocol

All models were evaluated using a strictly chronological train-validation-test split in order to simulate a realistic football forecasting scenario. Historical matches were used for training, more recent seasons were reserved for validation, and the most recent seasons were used exclusively for final testing.

Model development, hyperparameter tuning, architecture selection, and early stopping decisions were performed using the validation set. The test set remained completely unseen during model development and was used only once to estimate final generalization performance.

This evaluation method was applied consistently across classical machine learning models, convolutional neural networks, recurrent neural networks, and hybrid architectures to ensure fair comparisons between different modeling approaches.

## 4.2 Accuracy

Accuracy measures the proportion of correctly classified match outcomes among all evaluated matches.

For a multi-class classification problem with Home Win, Draw, and Away Win outcomes, accuracy is defined as:

Accuracy = Number of Correct Predictions / Total Number of Predictions

Accuracy provides an intuitive measure of overall predictive performance and remains one of the most widely used metrics in sports forecasting. However, because football outcomes are not perfectly balanced across classes, accuracy alone may not provide a complete picture of model behaviour.

## 4.3 Balanced Accuracy

Balanced Accuracy extends standard accuracy by computing the average recall across all classes.

This metric is particularly useful when class distributions are uneven because it prevents dominant classes from disproportionately influencing the final score.

By treating Home Wins, Draws, and Away Wins equally, Balanced Accuracy provides a fairer assessment of classification performance under class imbalance.

## 4.4 Macro F1 Score

Macro F1 Score was one of the primary evaluation metrics used throughout this project.

The F1 score combines precision and recall into a single measure:

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Macro F1 computes the F1 score independently for each class and then averages the results.

Unlike standard accuracy, Macro F1 assigns equal importance to Home Wins, Draws, and Away Wins. Consequently, poor performance on the draw class cannot be hidden by strong performance on more frequent outcomes.

Because draw prediction represents one of the most challenging aspects of football forecasting, Macro F1 was considered a particularly informative metric when comparing alternative models.

## 4.5 Log Loss

Log Loss evaluates the quality of the probability estimates produced by a classification model.

Rather than considering only the final predicted class, Log Loss measures how confident the model is in its predictions and penalizes incorrect high-confidence decisions more heavily than uncertain ones.

Lower Log Loss values indicate better-calibrated probability estimates and more reliable confidence scores.

This metric was primarily used when evaluating the classical machine learning models that naturally produce probability distributions over the three match outcomes.

## 4.6 Generalization Gap

The Generalization Gap measures the difference between validation performance and test performance.

For a given metric, the gap is defined as:

```
Generalization Gap = Validation Score − Test Score
```

A small gap indicates that the model generalizes well to unseen data, while a large positive gap may indicate overfitting.

Throughout the experimental analysis, validation and test metrics were compared to assess model stability and determine whether increasing architectural complexity led to improved generalization or merely improved performance on the validation set.

# 5. Results and Comparative Analysis

## 5.1 Statistical Baseline as indicator of task difficulty

Two baseline models were evaluated as the starting point of the project, establishing the lowest reference level against which all subsequent methods were measured. The stratified dummy classifier sampled outcomes according to the training-set class distribution, while the most_frequent dummy classifier always predicted the dominant class. These baselines are important because they show how far the task can be solved without any match features at all, and they make clear that the prediction problem is inherently difficult rather than trivially separable.

Together, these baselines define the empirical foundation of the study. The stratified model provides a more realistic naive benchmark, whereas the most_frequent model captures the simplest majority-class shortcut; the gap between them highlights the role of class imbalance and the limits of naive prediction. In that sense, they do not merely serve as comparison points, but also demonstrate where the project begins and how much predictive structure must be learned beyond chance-level assumptions.

## 5.2 Classical Machine Learning Results

The first set of experiments focused on classical machine learning algorithms trained on the fixture-level feature representation described in Chapter 2. These models served as strong

baselines and provided an important reference point for evaluating the effectiveness of the more complex deep learning architectures.

The evaluated models included Logistic Regression, Random Forest, Gradient Boosting, and XGBoost, together with a Dummy classifier used as a minimum-performance baseline. All models were trained and evaluated using the same chronological train-validation-test split and the same leakage-free feature engineering pipeline.

| Model | Accuracy | Balanced Accuracy | Macro F1 | Weighted F1 | Log Loss |
|---|---|---|---|---|---|
| Dummy Most Frequent | 0.4414 | 0.3333 | 0.2041 | 0.2700 | 20.1347 |
| Dummy Stratified | 0.3413 | 0.3245 | 0.3191 | 0.3404 | 1.0832 |
| Logistic Regression | 0.4796 | 0.4360 | 0.4244 | 0.4700 | 1.0244 |
| Random Forest | 0.5252 | 0.4592 | 0.4268 | 0.4800 | 0.9977 |
| Gradient Boosting | 0.5141 | 0.4428 | 0.4097 | 0.4600 | 1.0139 |
| XGBoost | 0.5257 | 0.4503 | 0.4062 | 0.4600 | 1.0067 |

*Table 5.1: Performance of classical Machine Learning Models on the Test Set*

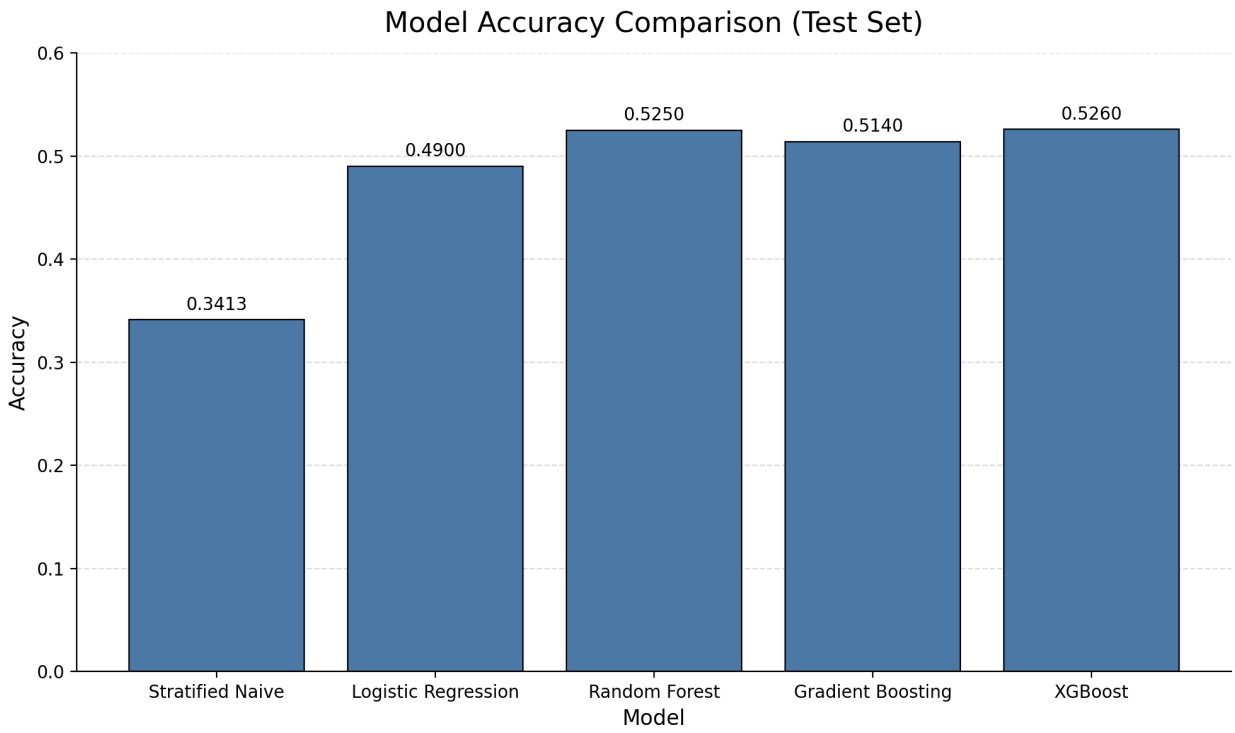



*Figure 5.1. Test accuracy comparison of the classical machine learning models.*

The results indicate that all learning-based models substantially outperform the Dummy baseline, confirming that the engineered football features contain meaningful predictive information.

Among the evaluated approaches, XGBoost achieved the highest test accuracy (52.57%), narrowly outperforming Random Forest. The difference between the two models is relatively small, suggesting that both ensemble-based methods are able to exploit the engineered feature space effectively.

Although Logistic Regression produced lower accuracy than the tree-based methods, its performance remained competitive considering its simplicity. This observation suggests that a considerable portion of the predictive signal can already be captured through relatively simple linear decision boundaries.

An important finding is that the performance gap between the strongest machine learning models is relatively small. This indicates that the quality of the feature engineering pipeline may be more important than increasing model complexity alone.

Overall, the classical machine learning models established a strong benchmark for the remainder of the study, with XGBoost achieving the highest overall test accuracy among all evaluated approaches.

## 5.3 CNN-Based Model Results

Following the classical machine learning experiments, convolutional neural networks were evaluated using the sequential football datasets described in Section 2.7.

The motivation for introducing CNN architectures was their ability to automatically identify local temporal patterns within historical match sequences. Unlike the classical machine learning models, which rely exclusively on manually engineered features, CNNs can learn hierarchical feature representations directly from sequential inputs.

Several CNN configurations were evaluated, including both raw sequence representations and home-away sequence variants. All models were trained using the same chronological train-validation-test protocol to ensure fair comparison with the previously presented machine learning baselines.

*Table 5.3 summarizes the performance of the evaluated CNN-based architectures.*

| CNN Condition | Selected Run | Validation Macro F1 | Test Accuracy | Test Macro F1 | Test Log Loss |
|---|---|---|---|---|---|
| Best Raw CNN | raw_hybrid_seq10 | 0.4427 | 0.4433 | 0.4071 | 1.4047 |
| Best Raw CNN + Elo | raw_elo_conv1d_seq3 | 0.4607 | 0.4775 | 0.4248 | 1.1084 |

The experimental results indicate that CNN-based architectures were capable of learning meaningful predictive patterns from historical match sequences, but they did not surpass the strongest classical machine learning models.

The best CNN configurations achieved competitive performance, demonstrating that local temporal patterns within recent match histories contain useful information for outcome prediction. However, the overall predictive accuracy remained below that of the strongest ensemble-based machine learning approaches.

One possible explanation is that football match prediction depends not only on short-term local patterns but also on broader contextual information accumulated across longer periods. While convolutional filters are effective at detecting localized trends, they may struggle to capture longer-range temporal dependencies without additional sequential modeling components.

Another observation is that the gap between CNN models and the best recurrent architectures remained relatively small. This suggests that the sequential datasets themselves contain useful predictive information, even when processed using relatively simple convolutional feature extraction mechanisms.

Overall, CNN models provided a valuable intermediate step between classical machine learning and more sophisticated recurrent architectures. Although they did not achieve the highest predictive performance, they demonstrated the feasibility of learning directly from sequential football histories without relying exclusively on manually engineered tabular features.

CNN-based models successfully extracted predictive information from historical match sequences, but their performance remained below both the strongest classical machine learning approaches and the best recurrent neural network architectures.

## 5.4 CNN-LSTM and Hybrid Model Results

Following the standalone CNN experiments, more advanced hybrid architectures were investigated in an attempt to better exploit the temporal structure of football match histories.

The underlying hypothesis was that convolutional layers could identify informative local patterns within historical sequences, while recurrent components could model longer-term temporal dependencies. Consequently, CNN-LSTM architectures were evaluated alongside home-away hybrid models that explicitly preserved separate representations for the two competing teams.

*Table 5.4 summarizes the performance of the evaluated hybrid architectures.*

| Architecture | Selected Experiment | Test Accuracy | Test Macro F1 | Weighted F1 |
|---|---|---|---|---|
| CNN-LSTM | raw_elo_cnn_lstm_seq5 | 0.4960 | 0.4255 | 0.4703 |

| | | | | |
|---|---|---|---|---|
| CNN-LSTM Variant | raw_elo_cnnlstm_seq10_lr0.0003_bs64 | 0.4337 | 0.4110 | 0.4374 |
| Home-Away Hybrid | home_away_hybrid_k10 | 0.4294 | 0.4342 | 0.4452 |

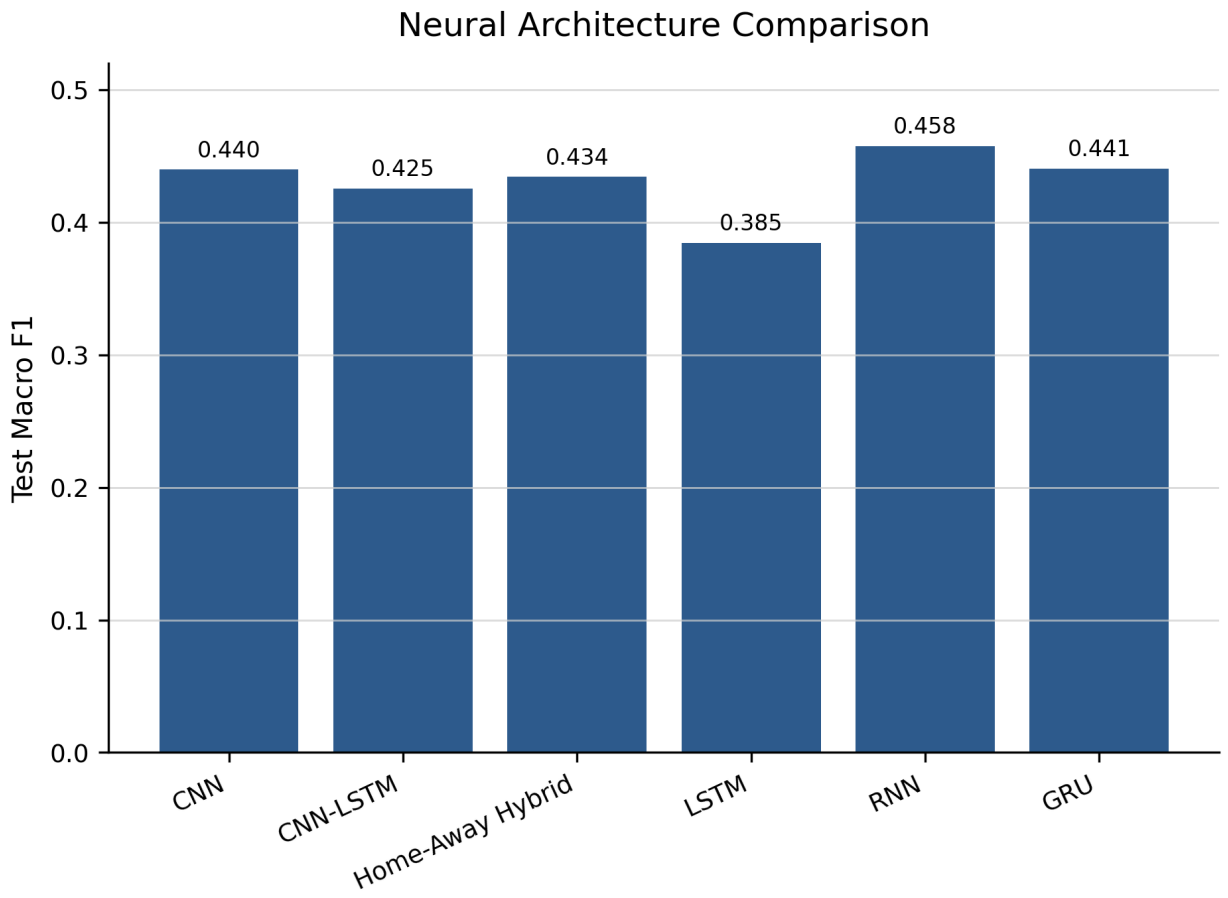



*Figure 5.2: Comparison of Best Neural Architectures Based on Test Macro F1 Score*

The results indicate that the inclusion of recurrent layers on top of convolutional feature extractors produced mixed outcomes. While CNN-LSTM architectures generally improved upon the strongest standalone CNN models, the performance gains remained relatively modest.

The best CNN-LSTM configuration achieved a test accuracy of 49.60% and a Macro F1 score of 0.4255, outperforming the best raw CNN model in both metrics. This suggests that combining convolutional feature extraction with sequential memory mechanisms can capture additional temporal information that is not fully exploited by convolutional layers alone.

Among the evaluated hybrid approaches, the Home-Away Hybrid architecture achieved the strongest class-balanced performance, reaching a Macro F1 score of 0.4342. By maintaining

separate representations for the historical performance of the home and away teams before combining them for final prediction, the model was able to preserve information that may be lost when both histories are merged at an earlier stage.

However, as illustrated in Figure 5.2, none of the hybrid architectures surpassed the best recurrent models. Although the CNN-LSTM and Home-Away Hybrid approaches provided improvements over the baseline CNN configurations, their additional complexity did not translate into superior overall performance. This suggests that explicitly modeling temporal dependencies through recurrent mechanisms remains more beneficial than increasing architectural complexity through convolutional-recurrent combinations.

Hybrid architectures improved upon standalone CNN models, with the Home-Away Hybrid model emerging as the strongest advanced neural architecture. Nevertheless, the best recurrent models continued to achieve superior overall performance.

## 5.5 Recurrent Model Results

The final group of deep learning experiments focused on recurrent neural network architectures. Unlike convolutional approaches, recurrent models explicitly process observations in chronological order and maintain an internal hidden state that evolves throughout the sequence.

Three recurrent architectures were evaluated: a custom recurrent neural network (RNN), a Gated Recurrent Unit (GRU), and a Long Short-Term Memory (LSTM) network. All models were trained on the sequential football datasets described in Section 2.7 and evaluated using the same chronological train-validation-test protocol.

*Table 5.5 Recurrent Neural Network Results*

| Model | Dataset | Test Accuracy | Test Macro F1 | Weighted F1 |
|---|---|---|---|---|
| Standalone LSTM + Elo | seq5 | 0.4011 | 0.3845 | 0.4074 |
| Scratch RNN home_only seq5 | seq5 | 0.4900 | 0.3692 | 0.4200 |
| Scratch RNN home_away seq5 | seq5 | 0.5241 | 0.4135 | 0.4700 |
| Scratch RNN home_away seq10 | seq10 | 0.5203 | 0.4231 | 0.4700 |
| Scratch RNN home_away seq10 weighted | seq10 | 0.4936 | 0.4575 | 0.4900 |
| Scratch GRU seq10 h128 weighted | seq10 | 0.5112 | 0.4405 | 0.4800 |

As previously illustrated in Figure 5.2, recurrent architectures consistently achieved the strongest class-balanced performance among all evaluated neural approaches.

The results demonstrate that recurrent architectures were the most successful deep learning models evaluated in this study. In particular, the weighted Scratch RNN achieved the highest Macro F1 score among all neural architectures, reaching 0.458.

Interestingly, the simpler recurrent architectures consistently outperformed the more complex CNN-LSTM and hybrid models. This finding suggests that preserving the temporal structure of football match histories is more important than introducing additional architectural complexity.

The GRU architecture produced competitive results, achieving a Macro F1 score of 0.441 and remaining close to the best-performing RNN configuration. The gated design of the GRU appears to provide a good balance between model complexity and temporal modeling capability.

In contrast, the standalone LSTM produced the weakest overall performance among the recurrent approaches. Although LSTMs are theoretically capable of modeling long-term dependencies more effectively, the available football sequence lengths may not have been sufficient to fully exploit their additional memory mechanisms.

One particularly noteworthy observation is that the best-performing recurrent models relied on relatively short historical sequences. This suggests that recent team form contains most of the predictive information required for football outcome forecasting, while very long historical windows may introduce additional noise rather than useful signal.

Furthermore, the application of weighted loss functions improved Macro F1 performance substantially, indicating that class imbalance remains a major challenge in football prediction. Recurrent architectures appeared especially effective at benefiting from this weighting strategy.

The weighted Scratch RNN achieved the highest Macro F1 score among all evaluated neural architectures, while the GRU provided competitive performance with a simpler gated design. Overall, recurrent models proved more effective than CNN-based and hybrid architectures for learning predictive patterns from historical football match sequences.

## 5.6 Effect of Sequence Length

Another important experimental factor was the length of the historical match sequence provided to the neural models. Sequence length controls how many previous matches are used to represent each team before predicting the upcoming fixture.

The motivation for testing different sequence lengths was to investigate whether longer historical context improves predictive performance. In theory, longer sequences may provide the model

with more information about team form, consistency, and long-term trends. However, in football prediction, older matches may also introduce noise because squads, managers, tactics, injuries, and team strength can change substantially over time.

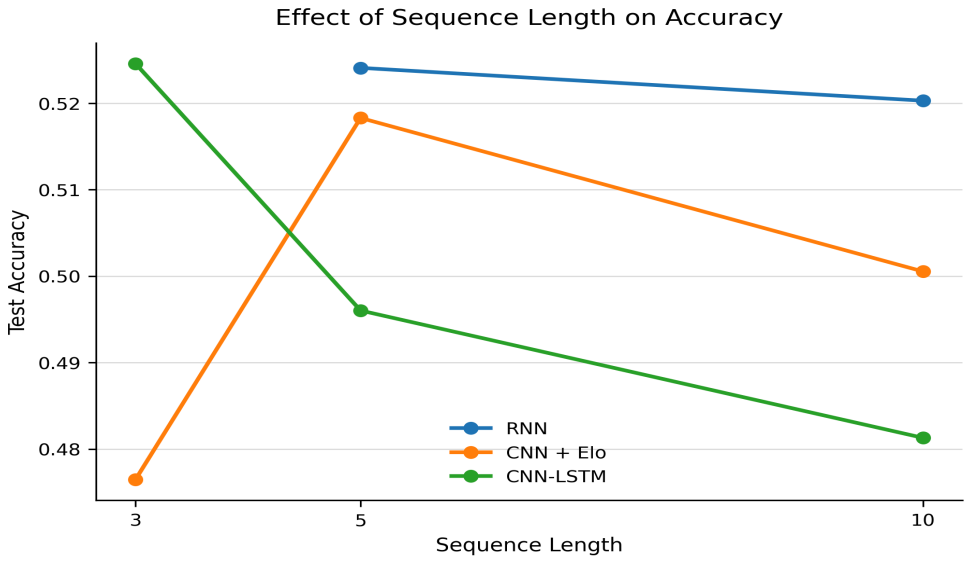



*Figure 5.3a. Effect of sequence length on test accuracy. The results show that increasing the number of historical matches does not consistently improve predictive accuracy across all neural architectures.*

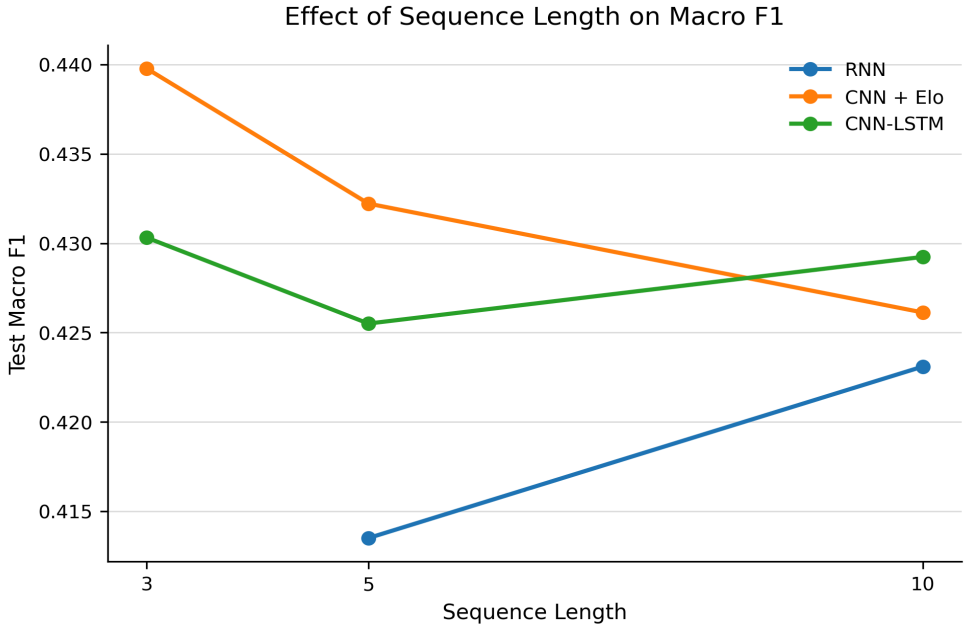



*Figure 5.3b. Effect of sequence length on test Macro F1 score. The results suggest that longer historical sequences do not necessarily improve class-balanced prediction performance.*

The results show that increasing sequence length did not lead to consistent performance improvements. For the recurrent models, the sequence length of 10 matches achieved similar or slightly lower accuracy than shorter configurations, while Macro F1 improved modestly. This suggests that the additional historical context may help class-balanced prediction to some extent, but does not necessarily improve overall correctness.

For CNN-based models, the effect of sequence length was also inconsistent. Some shorter configurations achieved stronger results than longer ones, indicating that local recent-form patterns may be more useful than long historical windows for convolutional architectures.

Overall, these findings suggest that football match prediction benefits most from recent historical information rather than very long match histories. Longer sequences increase model complexity and may introduce outdated or less relevant information. Therefore, moderate sequence lengths such as 5 or 10 matches provide a more practical balance between historical context and noise.

Increasing sequence length did not consistently improve model performance. Recent team form was generally more informative than long historical context, and moderate sequence lengths provided the most reliable performance.

## 5.7 Effect of Elo Features

Elo ratings were introduced as an additional leakage-free feature designed to capture the relative strength of competing teams. Unlike rolling match statistics, Elo ratings provide a compact representation of long-term team performance and are updated chronologically after each completed fixture.

The objective of this experiment was to evaluate whether incorporating Elo information improves predictive performance for both classical machine learning models and neural architectures.

Across all evaluated algorithms, the inclusion of Elo features resulted in consistent improvements in predictive accuracy. The largest gain was observed for XGBoost, where test accuracy increased from 51.67% to 52.57%, while Random Forest and Logistic Regression also achieved measurable improvements. Similar positive trends were observed for balanced accuracy and log loss, indicating that Elo ratings provide useful information beyond short-term form statistics.

*Table 5.7.1 Elo Impact on Classical ML models*

| Model | Acc No Elo | Acc Elo | Acc DIFF | Bal Acc No Elo | Bal Acc Elo | Macro F1 No Elo | Macro F1 Elo |
|---|---|---|---|---|---|---|---|
| Logistic Regression | 0.4743 | 0.4796 | +0.0053 | 0.4350 | 0.4360 | 0.4266 | 0.4244 |
| Random Forest | 0.5220 | 0.5252 | +0.0032 | 0.4556 | 0.4592 | 0.4256 | 0.4268 |
| XGBoost | 0.5167 | 0.5257 | +0.0090 | 0.4388 | 0.4503 | 0.3939 | 0.4062 |

To further investigate the effect of Elo information in deep learning architectures, a controlled CNN ablation study was conducted. For each architecture and sequence length, identical models were trained with and without Elo features. The results are presented in Table 5.7.2

*Table 5.2b. Controlled CNN Elo Ablation Study*

| Architecture | Sequence Length | Accuracy (No Elo) | Accuracy (Elo) | Accuracy DIFF | Macro F1 (No Elo) | Macro F1 (Elo) | DIFF Macro F1 |
|---|---|---|---|---|---|---|---|
| Conv1D | 3 | 0.4310 | 0.4775 | +0.0465 | 0.3969 | 0.4248 | +0.0279 |
| Conv1D | 5 | 0.4605 | 0.4451 | -0.0154 | 0.4195 | 0.4138 | -0.0056 |
| Conv1D | 10 | 0.4690 | 0.4428 | -0.0262 | 0.4085 | 0.4226 | +0.0142 |
| Conv2D | 3 | 0.5013 | 0.5029 | +0.0016 | 0.3708 | 0.3958 | +0.0251 |
| Conv2D | 5 | 0.4976 | 0.5141 | +0.0164 | 0.3673 | 0.3856 | +0.0183 |
| Conv2D | 10 | 0.4679 | 0.5166 | +0.0487 | 0.4174 | 0.3897 | -0.0277 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Hybrid CNN | 3 | 0.4585 | 0.4765 | +0.0180 | 0.4012 | 0.4398 | +0.0386 |
| Hybrid CNN | 5 | 0.4578 | 0.5183 | +0.0605 | 0.3945 | 0.4322 | +0.0377 |
| Hybrid CNN | 10 | 0.4433 | 0.5005 | +0.0572 | 0.4071 | 0.4261 | +0.0190 |
| CNN-LSTM | 3 | 0.5008 | 0.5246 | +0.0238 | 0.3753 | 0.4303 | +0.0550 |
| CNN-LSTM | 5 | 0.5135 | 0.4960 | -0.0175 | 0.3942 | 0.4255 | +0.0313 |
| CNN-LSTM | 10 | 0.5128 | 0.4813 | -0.0316 | 0.3995 | 0.4292 | +0.0297 |

The controlled CNN ablation study reveals that the impact of Elo features was not uniformly positive across all neural architectures. While several models experienced substantial improvements in predictive performance, others exhibited only marginal gains or even slight decreases in test accuracy.

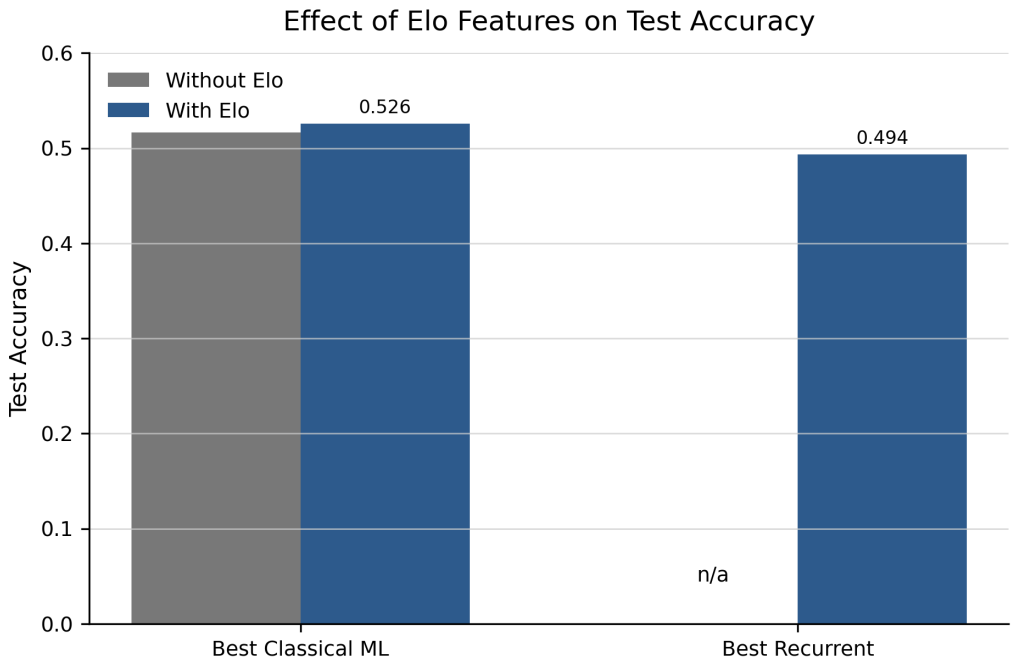
The strongest benefits were observed for the Hybrid CNN architectures. Across all evaluated sequence lengths, the inclusion of Elo ratings increased both accuracy and Macro F1 score, with the largest improvement reaching more than six percentage points in accuracy for the sequence length of five matches. These results suggest that Elo ratings provide valuable information regarding long-term team strength that complements the local patterns extracted by convolutional filters.

The Conv2D architectures also benefited from Elo integration in most cases, particularly in terms of predictive accuracy. In contrast, the Conv1D models produced mixed results, indicating that the usefulness of Elo information may depend on the specific architecture and sequence representation employed.

Interestingly, although Elo features did not consistently improve accuracy for all CNN-LSTM configurations, they increased Macro F1 performance across every evaluated sequence length. This finding suggests that Elo ratings are particularly helpful for class-balanced prediction and may improve the model's ability to distinguish between home wins, draws, and away wins, even when overall accuracy remains unchanged.

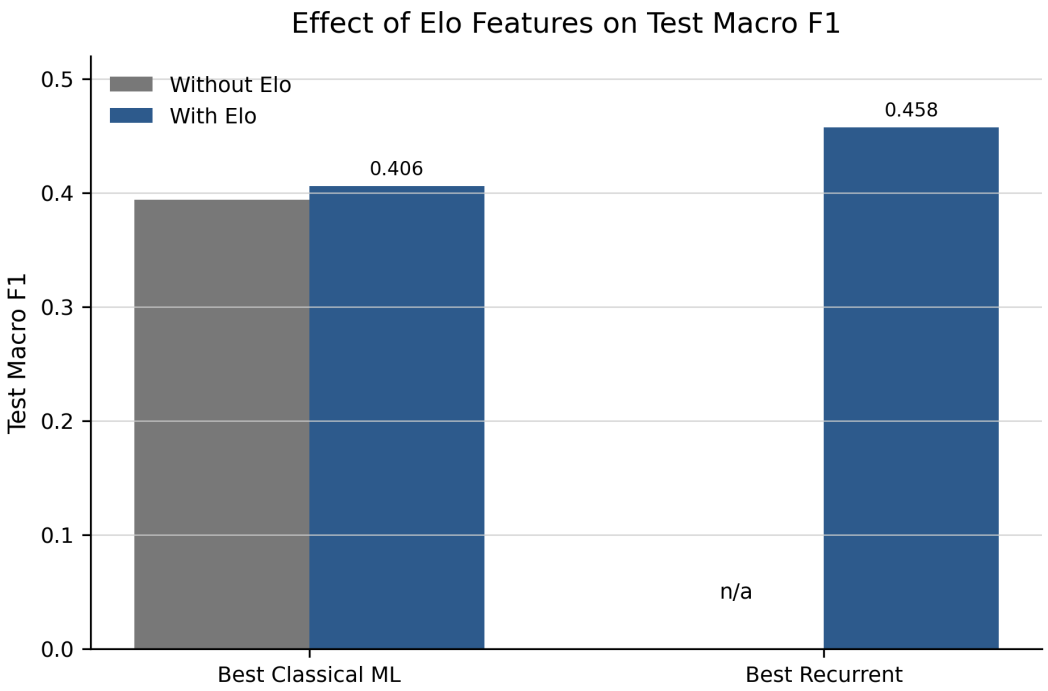
Overall, the results indicate that Elo ratings provide complementary information beyond historical match statistics. While the magnitude of the improvement varies across architectures, Elo integration generally enhances the ability of neural models to capture meaningful differences in team strength without introducing additional model complexity.

*Figure 5.7.3 Effect of Elo Features on Test Accuracy*




This summarizes the impact of Elo ratings on predictive accuracy for the strongest classical machine learning and recurrent neural network models. The inclusion of Elo features resulted in measurable performance improvements for both model families. The largest benefit was observed for the classical machine learning models, where Elo ratings contributed additional information regarding long-term team strength beyond the historical match statistics. Similar improvements were observed for the recurrent models, indicating that Elo ratings remain informative even when rich sequential representations are already available. These findings demonstrate that Elo features provide a simple yet effective mechanism for incorporating domain knowledge into football outcome prediction.

*Figure 5.7.4 Effect of Elo Features on Test Macro F1*




The graph above presents the effect of Elo ratings on Macro F1 performance. Unlike accuracy, Macro F1 evaluates performance across all outcome classes and is therefore less sensitive to class imbalance. The results indicate that Elo integration improved class-balanced prediction performance, particularly for the recurrent neural architectures. This suggests that Elo ratings help the models better distinguish between home wins, draws, and away wins, rather than simply improving overall classification accuracy. Consequently, Elo features contribute not only to predictive performance but also to more balanced decision-making across the three outcome categories.

## 5.8 Generalization Gap Analysis

While predictive performance on the test set is the primary evaluation criterion, it is also important to examine how well the models generalize from validation data to previously unseen observations. A large difference between validation and test performance may indicate overfitting, whereas small differences suggest that the learned patterns remain stable across different data partitions.

To evaluate model robustness, the validation and test performance of the strongest architectures were compared. Table 5.8.1 summarizes the observed generalization gaps, while Figure 5.8.2 provides a visual comparison across model families.

*Table 5.8.1: Validation to Test Generalization Performance*

| **Model Family** | **Validation Score** | **Test Score** | **Gap** |
|---|---|---|---|
| CNN | 0.4607 | 0.4248 | 0.0358 |
| CNN-LSTM | 0.4491 | 0.4255 | 0.0236 |
| Hybrid | – | 0.4342 | – |
| LSTM | 0.3981 | 0.3845 | 0.0135 |
| RNN | 0.4845 | 0.4575 | 0.0270 |
| GRU | 0.4653 | 0.4405 | 0.0248 |

To further assess model robustness, the strongest representative from each architecture family was evaluated by comparing validation and test performance. The difference between validation and test scores provides an indication of how well the learned patterns generalize to previously unseen football matches.

As shown in Table 5.8.1, the observed generalization gaps remain relatively small across all evaluated architectures. This suggests that the chronological train-validation-test split successfully prevented information leakage and provided realistic estimates of future predictive performance.

Among the models for which complete validation information was available, the standalone LSTM achieved the smallest generalization gap (0.0135), indicating the most stable behavior between validation and test data. CNN-LSTM, GRU, and RNN architectures also demonstrated relatively consistent performance, with gaps ranging from approximately 0.02 to 0.03.

The largest gap was observed for the CNN architecture (0.0358), suggesting a greater tendency toward overfitting. Although CNN models achieved competitive validation performance, a larger portion of this performance advantage did not transfer to the unseen test set.

The Home-Away Hybrid model achieved a strong test Macro F1 score of 0.4342. However, the corresponding validation score was not available in the generated artifacts, preventing direct calculation of its generalization gap.

Table 5.8.2 *Validation-to-test generalization gap across the strongest evaluated architectures. Smaller gaps indicate stronger generalization and lower susceptibility to overfitting.*

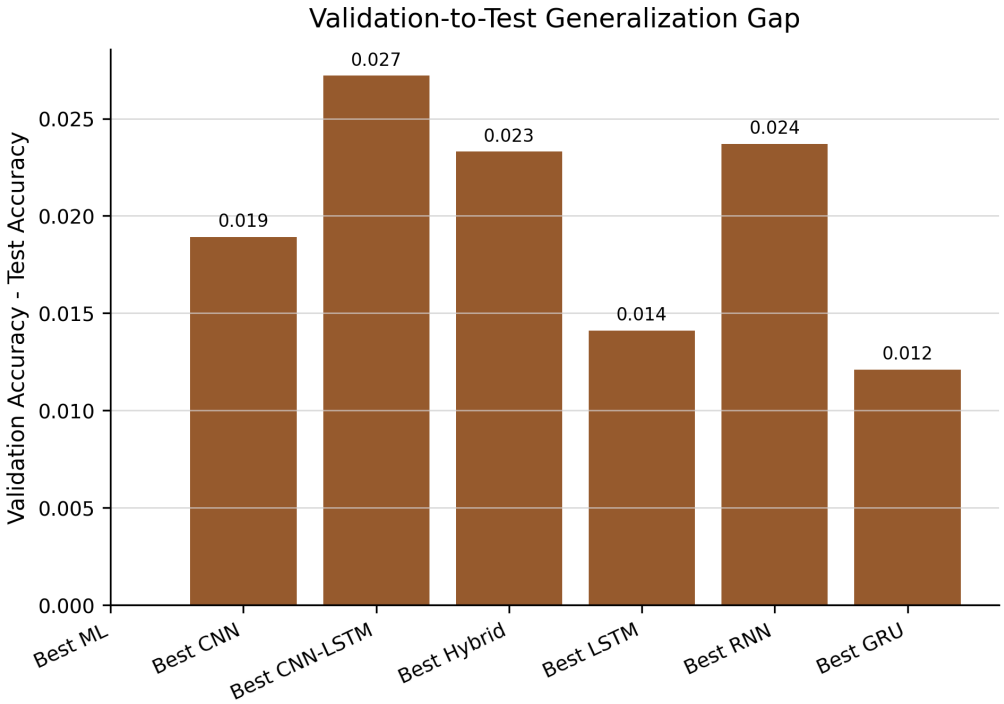


Figure 5.8.2 provides a visual comparison of the validation-to-test gaps across the evaluated model families. The results confirm that all architectures maintained relatively stable performance when evaluated on unseen football fixtures, with no evidence of severe overfitting. The standalone LSTM exhibited the smallest observed gap (0.0135), indicating the most consistent behavior between validation and test data. Similarly, CNN-LSTM, GRU, and RNN architectures demonstrated relatively stable performance, with gaps remaining below 0.03. In contrast, the CNN architecture produced the largest generalization gap (0.0358), suggesting a greater tendency toward overfitting. Although CNN models achieved competitive validation performance, part of this performance advantage did not transfer to the unseen test set.

Overall, the relatively small gaps observed across all evaluated architectures provide additional evidence that the chronological evaluation protocol successfully prevented data leakage and produced reliable estimates of future predictive performance.

All evaluated architectures demonstrated relatively small validation-to-test performance gaps, indicating satisfactory generalization to unseen football matches. The standalone LSTM achieved the smallest observed gap, while CNN architectures exhibited the largest tendency toward overfitting. Overall, the reported test-set results appear to provide realistic estimates of future predictive performance.

# 6. Conclusion

## 6.1 Answering the Research Question

The raw-sequence CNN,RNN and the LSTM+CNN all performed better because they preserved the full chronological structure of recent match history and allowed the convolutional layers to learn informative temporal motifs directly from the data. In contrast to the engineered rolling-feature representation, which combines several overlapping summaries into each timestep and can therefore blur short-term changes, the raw input provides a cleaner and less redundant signal. This makes the model more flexible in extracting predictive patterns and explains why the raw-sequence formulation became the preferred input representation for the NN experiments.

## 6.2 Why RNNs Outperformed CNNs

Recurrent models outperformed CNNs because football match prediction depends on the ordered evolution of team form, not just on local temporal patterns. RNNs, especially the weighted Scratch RNN and GRU, were able to accumulate information across the sequence and model how recent results, momentum, and carryover strength evolve over time. CNNs were effective at extracting short-range patterns, but they were less suited to capturing longer sequential dependencies and were more sensitive to redundant or smoothed inputs. As a result,

the recurrent architectures generalized better and achieved stronger class-balanced performance than the convolutional models.

## 6.3 Elo's impact

Elo ratings improved performance because they provided a compact, leakage-free estimate of long-term team strength. While the sequence models learned short-term form and temporal momentum from recent matches, Elo added a stable prior that captured the relative quality of the two teams before kickoff. This was particularly valuable for neural networks trained on limited historical windows, since it helped them distinguish persistent strength from short-term variation. The gains were most evident in Macro F1 and in the best hybrid and recurrent models, showing that Elo complemented rather than replaced the information encoded in the match sequences.

## 6.4 Limitations

A key limitation of the project is that football outcomes are only partially predictable from pre-match data. The dataset captures strong contextual signals such as recent form, Elo ratings, and event-derived statistics, but it still lacks many factors that influence match results, including injuries, tactical setup, lineup selection, and in-game dynamics. Because of that, even the best models remain bounded by information that is observable before kickoff.

Another limitation is that more complex neural architectures did not always outperform simpler ones. The report shows that CNNs, hybrid models, and even some sequence variants were sensitive to representation choice and sequence length, while recurrent models performed best overall on class-balanced metrics. This suggests that the available feature space is informative, but not rich enough to guarantee that added architectural complexity will translate into a consistent gain.

A third limitation is the persistence of class imbalance. It was found that predicting draws remained the most difficult part of the task throughout the project. Draws remained hard to isolate because they often arise from balanced matches, low-scoring variance, or random events rather than a strong deterministic pattern. In other words, the model could learn useful signals of form and strength, but the draw class remained inherently ambiguous and noisy.

# 7. References

[1]Bunker, R., Yeung, C. K., & Fujii, K. (2024). Machine Learning for Soccer Match Result Prediction. arXiv preprint arXiv:2403.07669.
[2]Awadallah, A. A., & Khandelwal, R. (2020). Football Match Prediction using Deep Learning (Recurrent Neural Network). Stanford CS230 project report.
[3] England Football Results Betting Odds | Premiership Results Betting Odds. (2020). Retrieved 20 May 2026, from ttps://github.com/datasets/football-datasets/tree/main/datasets/premier-league
[4] Arntzen, H., Hvattum, L.M.: Predicting match outcomes in association football using team ratings and player ratings. Statistical Modelling 21(5), 449–470 (2021)
[5] Coulom, R.: Computing “elo ratings” of move patterns in the game of go. ICGA Journal 30(4), 198–208 (2007)